

UNIVERSIDADE ESTADUAL DE MONTES CLAROS – UNIMONTES
CENTRO DE CIÊNCIAS EXATAS E TECNOLÓGICAS – CCET
DEPARTAMENTO DE CIÊNCIAS DA COMPUTAÇÃO – DCC

EXERCÍCIO PROVA 2

RAMON LOPES DE QUEIROZ

MONTES CLAROS – MG
JUNHO / 2026

RAMON LOPES DE QUEIROZ

EXERCÍCIO PROVA 2

Atividade avaliativa apresentada para atendimento de requisito parcial para aprovação na disciplina AEDS II do Curso de Graduação em Bacharelado em Sistemas de Informação – 2º período

Professor: Dr. Heveraldo Rodrigues de Oliveira

MONTES CLAROS – MG

JUNHO / 2026



Exercício II

Nome: _____ Data: 18/06/2026

1. A operação de busca é encontrada com muita frequência em aplicações computacionais, sendo, portanto, importante estudar estratégias distintas para efetuar a busca. Se tivermos muitos itens cadastrados, o método de busca deve ser eficiente, caso não seja, a busca poderá ser muito demorada e inviável. Com relação aos algoritmos de busca, verifique as seguintes afirmativas:

I – A busca linear ou busca sequencial tem ordem $O(n^2)$.

II – A busca binária tem um custo de pesquisa da ordem de $O(\log n)$ no pior caso.

III – A busca binária tem ordem $O(\log n)$ mesmo com dados não ordenados.

IV – A árvore binária de busca permite inserir, retirar e buscar itens com um custo que é igual a altura da árvore no pior caso.

V – A busca em uma árvore de busca é de ordem $O(n)$, caso a árvore esteja balanceada.

Justifique as afirmativas falsas:

2. Mostre os subvetores gerados pela ordenação do quicksort para o vetor abaixo.

2, 4, 6, 8, 10, 12, 1, 3, 5, 7, 9, 11 Considere como pivô o mais à direita de cada subvetor.

3. Desenhe a árvore binária de busca pela inserção dos dados na seguinte ordem:

5, 12, 18, 20, 27, 35, 99, 33, 87, 77, 45, 76, 73, 89, 34, 44, 1, 2, 3

4. A busca binária pode ser implementada de forma recursiva. Complete as lacunas no programa abaixo, que implementa a **Busca Binária Recursiva**:

```
int busca_bin_rec(int vet[], int ini, int fim, int elem) {  
    if ( _____ ) { // Caso de parada: elemento não encontrado  
        return -1;  
    }  
  
    int meio = _____;  
  
    if ( _____ ) { // Encontrou o elemento  
        return meio;  
    }  
    else if ( _____ ) { // Busca na metade esquerda  
        return busca_bin_rec(vet, ini, meio - 1, elem);  
    }  
    else { // Busca na metade direita  
        return _____;  
    }  
}
```

5. Considere o programa abaixo que implementa uma estrutura de árvore binária de busca:

```
#include <stdlib.h>
#include <stdio.h>
struct abb {
    int Info;
    struct abb* esq;
    struct abb* dir;
};
typedef struct abb Abb;

Arv* abb_cria(void) {
    return NULL;
}

void abb_imprime(Abb* a) {
    if (a != NULL) {
        printf("%d\n", a->Info);
        abb_imprime(a->esq);
        abb_imprime(a->dir);
    }
}

int max(int a, int b) {
    return a > b ? a : b;
}

int abb_Altura(Abb* a) {
    if (a == NULL) return 0;
    return 1 + max(abb_Altura(a->esq), abb_Altura(a->dir));
}
```

Responda:

- a) Escreva uma função recursiva `abb_BuscaAltura` que retorne a altura do nó com o valor buscado ou -1 caso o valor não seja encontrado:

```
int abb_BuscaAltura(Abb *a, int v);
```

- b) Mostre a ordem de impressão dos dados da árvore da questão 3 (três), caso seja executada a função `abb_imprime` implementada acima.

- c) Crie uma função `abb_MaiorValor` que retorne o **maior valor** armazenado em `Info`. A função deve percorrer apenas as ramificações necessárias para se encontrar o maior valor em uma ABB (assuma que a árvore é uma ABB válida):

```
int abb_MaiorValor(Abb *a);
```

Exercício prova II -

I - Falso

II - Verdadeira

III - Falso

IV - Verdadeira

V - Falso

I. A busca linear percorre o vetor de for a direita, mesmo no pior caso é de ordem linear $O(n)$.

III - De ordens distintos não-ordenados, o algoritmo não comete erros e falhas.

V. No caso de balanceamento, o número é $O(\log n)$.

2. $[2, 4, 6, 8, 10, 12, 1, 3, 5, 7, 9, 11] \rightarrow$ Pivô 11, isolado à dir.

$2, 4, 6, 8, 10, 1, 3, 5, 7, 9, 11, 12 \rightarrow$ Pivô novo = 9

$2, 4, 6, 8, 10, 1, 3, 5, 7, 9$

9 fixo, isolado à dir.

$2, 4, 6, 8, 1, 3, 5, 7, 9, 10 \rightarrow$ Pivô 7, 7 fixo, 8 isolado à dir.

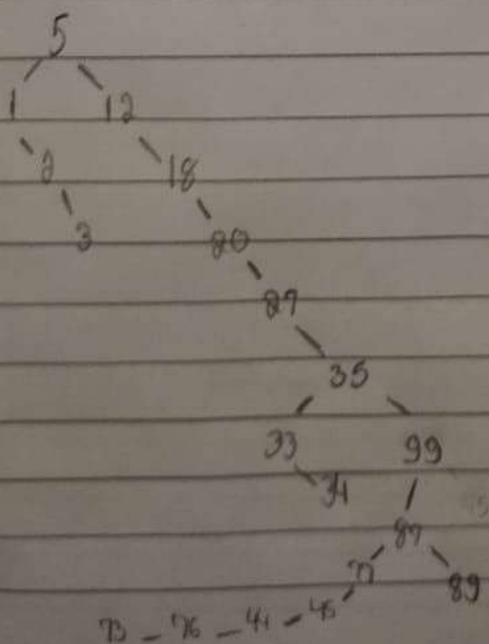
$2, 4, 6, 1, 3, 5 \rightarrow$ Pivô 5, 5 fixo, 6 isolado à dir.

$2, 4, 1, 3 \rightarrow$ Pivô 3, 3 fixo, 4 isolado à dir.

$2, 1 \rightarrow$ Pivô 1, 1 fixo, 2 isolado à direita

Result = $1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12$

3.



4. $Ini > fim$ | $(ini + fim) / 2$ | $next[meio] == elem$ | $next[meio] > elem$ |
 busca - meio = rec(next, meio + 1, fim, elem).

5a. `int altura - no (Arb * a) {`
 `if (a == NULL) return 0;`
 `int alt - esq = altura - no(a -> esq);`
 `int alt - dir = altura - no(a -> dir);`
 `return 1 + (alt - esq > alt - dir ? alt - esq : alt - dir);`
`}`

`int arb - BuscaAltura (Arb * a, int re) {`
 `if (a == NULL) return -1;`
 `if (a -> Info == re) return altura - no(a);`
 `if (re < a -> Info) {`
 `return arb - BuscaAltura(a -> esq, re);`
 `} else {`
 `return arb - BuscaAltura(a -> dir, re);`
 `}`
`}`

b. 5, 1, 2, 3, 19, 20, 27, 35, 33, 34, 99, 87, 77, 45, 44, 76, 73, 89.

c. `int arb - MaisValor (Arb * a) {`
 `if (a == NULL) return -1;`
 `if (a -> dir == NULL) return a -> Info;`
 `return arb - MaisValor(a -> dir);`
`}`